

CS599 期末大作业报告

面向菜谱问答与个性化饮食推荐的多智能体系统

课程名称	企业级应用软件设计与开发
课程代码	50120224001 / CS599
项目名称	SmartRecipe Multi-Agent System
方向	方向一：Agentic AI 原生开发
学号	2025302928
姓名	苏笑省
专业	计算机技术
指导教师	戚欣
提交日期	2026 年 6 月 22 日
GitHub 仓库	cs599-project-SmartRecipe

目录

- 一、选题背景与设计思想.....4
 - 1.1 问题背景4
 - 1.2 项目价值4
 - 1.3 技术路线4
- 二、Specs 规格文档.....5
 - 2.1 Product Spec.....5
 - 2.1.1 产品定位.....5
 - 2.1.2 用户角色.....5
 - 2.1.3 核心功能需求.....5
 - 2.1.4 非功能需求.....6
 - 2.2 Architecture Spec.....7
 - 2.2.1 总体架构7
 - 2.2.2 数据架构7
 - 2.2.3 Agent 状态设计.....7
 - 2.3 API Spec.....8
 - 2.3.1 文本聊天接口.....8
 - 2.3.2 图片聊天接口.....9
 - 2.3.3 健康检查与调试接口9
- 三、系统架构与设计10
 - 3.1 多 Agent 交互流程.....10
 - 3.2 检索设计10
 - 3.3 存储设计10
 - 3.4 安全与工程规范.....12

四、关键实现与代码展示.....	12
4.1 FastAPI 应用入口	12
4.2 LangGraph 工作流	12
4.3 Router Agent	13
4.4 MySQL 与 Neo4j 数据建模	13
4.5 PDF 菜谱管线	14
4.6 HyDE 与混合检索	14
4.7 AI IDE 使用说明	15
五、测试与评估	15
5.1 自动化测试	15
5.2 测试覆盖范围	16
5.3 功能验证用例	16
5.4 评估脚本	17
5.5 可观测性	19
六、系统升级与扩展.....	19
6.1 当前不足	19
6.2 下一阶段计划	19
6.3 AI 能力演进路径.....	20
七、课程总结	20
7.1 个人收获	20
7.2 工程思维转变	20

一、选题背景与设计思想

1.1 问题背景

日常饮食推荐看似简单，实际涉及多个维度：用户口味、忌口、过敏、健康目标、菜谱知识、食材关系、营养约束以及多轮对话上下文。传统菜谱应用通常以关键词搜索和静态分类为主，存在以下不足：

1. 检索方式偏机械，只能按菜名、食材或分类搜索，难以理解“低脂晚餐”“高蛋白少油”“家里只有鸡蛋和番茄”这类自然语言需求。
2. 缺少跨会话记忆，用户每次都要重复说明偏好、过敏、忌口和饮食目标。
3. 结构化数据、图谱关系和文档知识割裂，无法同时利用 SQL 表、Neo4j 图谱、PDF 菜谱书和本地菜谱库。
4. 多模态能力不足，不能根据菜品图片识别结果继续检索相关菜谱。
5. 检索结果缺少可信度保护，容易把相似菜谱误当作目标菜谱回答。

本项目选择“Agentic AI 原生开发”方向，从零构建一个具备实际价值的智能菜谱助手 SmartRecipe。系统将 FastAPI、LangGraph、多智能体协作、RAG、结构化数据库、图数据库、Redis 记忆、视觉模型和评估脚本整合为完整工程闭环。

1.2 项目价值

SmartRecipe 的目标不是做一个单轮聊天机器人，而是实现一个可扩展、可评估、可落地的企业级智能应用原型。它面向以下典型场景：

- 用户询问具体菜谱做法，例如“老醋花生米怎么做”。
- 用户提出约束推荐，例如“推荐低脂、高蛋白、少油的晚餐”。
- 用户希望系统记住长期偏好，例如“不吃香菜，对虾过敏”。
- 用户询问结构化统计，例如“热量最低的五道菜是什么”。
- 用户询问图谱关系，例如“鸡胸肉通常和哪些食材搭配”。
- 用户上传菜品图片，系统识别菜品后给出介绍和相似菜谱。
- 用户导入扫描版 PDF 菜谱书，系统 OCR、分块、建立索引并用于问答。

1.3 技术路线

本项目以 Agentic AI 为核心，把“自然语言理解、工具调用、状态管理、记忆、检索、结构化查询、图谱推理、视觉识别、评估”拆分到不同 Agent 和服务层中。核心技术包括：

- SDD 规格驱动开发：先定义 Product Spec、Architecture Spec、API Spec，再实现代码。
- LangGraph 状态机：统一编排 Safety、Preference、Router、Recipe、SQL、Cypher、Fusion、Vision、Rerank、Answer 等 Agent。
- Agentic RAG：融合 BM25、FAISS HNSW、BGE embedding、HyDE、Query Rewrite、Cross-Encoder rerank。
- 多源数据增强：MySQL 存结构化菜谱，Neo4j 存菜谱关系图谱，Redis 存会话记忆，FAISS 存文档向量索引。
- 多模态输入：/chat/image 接收图片，Vision Agent 将识别结果接入现有 RAG 与回答链路。
- 可观测性与评估：提供日志、Debug API、测试集、消融实验和 pytest 自动化测试。

二、Specs 规格文档

2.1 Product Spec

2.1.1 产品定位

SmartRecipe 是一个面向中文菜谱问答、个性化饮食推荐和菜谱知识管理的多智能体系统。系统通过自然语言、图片和文档三类输入，帮助用户完成菜谱检索、做法查询、营养建议、食材替换、关系分析和 PDF 菜谱书问答。

2.1.2 用户角色

用户角色	需求
普通家庭用户	快速查询菜谱做法，根据手头食材获得推荐
健康饮食用户	按低脂、高蛋白、控糖、少盐等目标筛选菜谱
有忌口用户	系统记住过敏、忌口和不喜欢的食材
菜谱资料整理者	将 PDF 菜谱书 OCR、结构化、分块并纳入检索
开发/评测人员	通过 Debug API、日志和测试脚本验证 Agent 行为

2.1.3 核心功能需求

编号	功能	说明	优先级
F1	文本问答	支持菜谱推荐、做法查	P0

		询、食材替换、营养问答	
F2	多 Agent 路由	根据意图分发到 Recipe、Nutrition、 SQL、Cypher、Fusion 等 Agent	P0
F3	RAG 检索	支持本地菜谱 JSON、 BM25、FAISS HNSW 和 PDF 文档 RAG	P0
F4	记忆机制	基于 session_id 存储多轮 对话、偏好、过敏和忌口	P0
F5	数据库查询	MySQL Text2SQL 查询 结构化菜谱数据	P1
F6	图谱查询	Neo4j Text2Cypher 查 询食材、标签、目标和约 束关系	P1
F7	多源融合	Fusion Agent 合并 RAG、SQL、Cypher、 GraphRAG 结果	P1
F8	图片识别	上传菜品图片并转为可检 索语义	P1
F9	PDF 菜谱导入	OCR、菜谱结构化分块、 建立 FAISS 索引、导入 MySQL/Neo4j	P1
F10	评估与调试	pytest、评估脚本、 Debug API、日志和消融 实验	P0

2.1.4 非功能需求

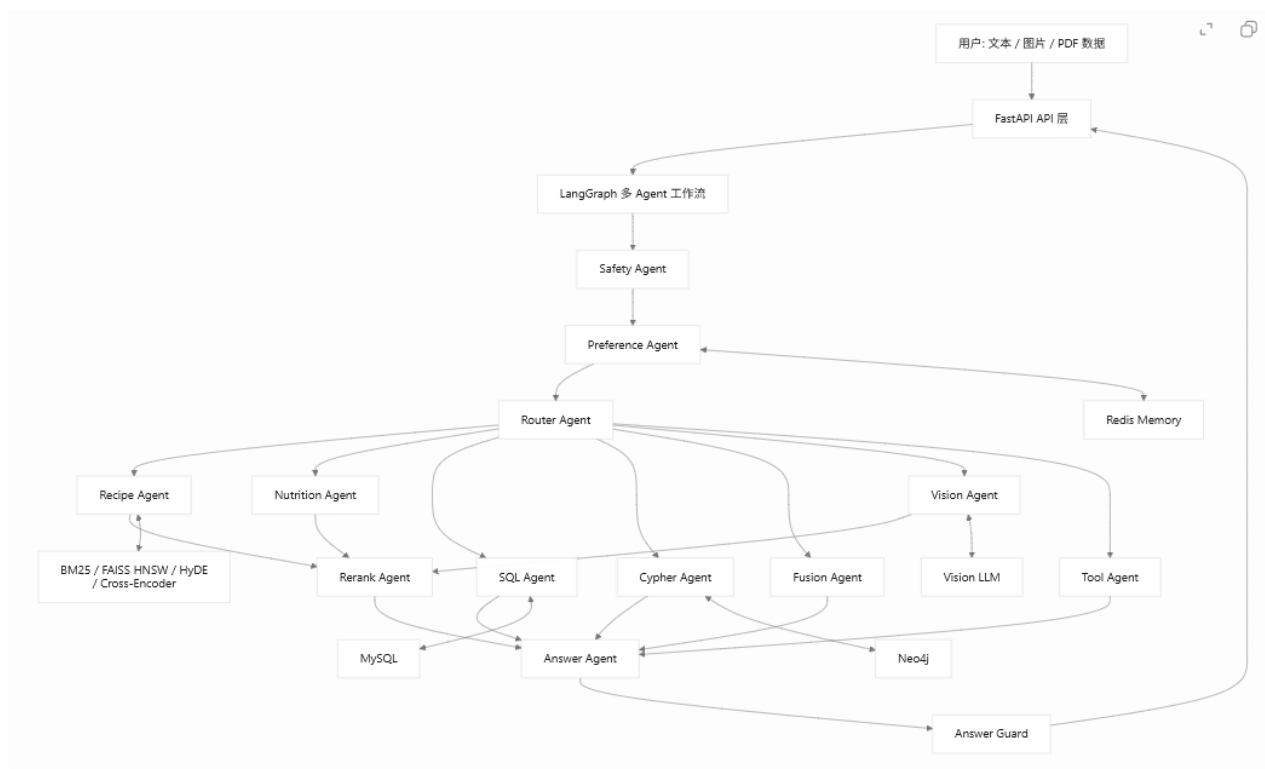
- 安全性：API Key 通过 .env 管理，不硬编码；SQL/Cypher 只允许只读查询。
- 可维护性：Agent、services、tools、scripts 分层清晰。

- 可扩展性：新增 Agent、工具或数据源时不破坏主链路。
- 可观测性：每个 Agent 节点输出日志，Debug API 可查看数据源和能力状态。
- 可复现性：本机 conda 运行应用，Docker 运行 MySQL、Redis、Neo4j。

2.2 Architecture Spec

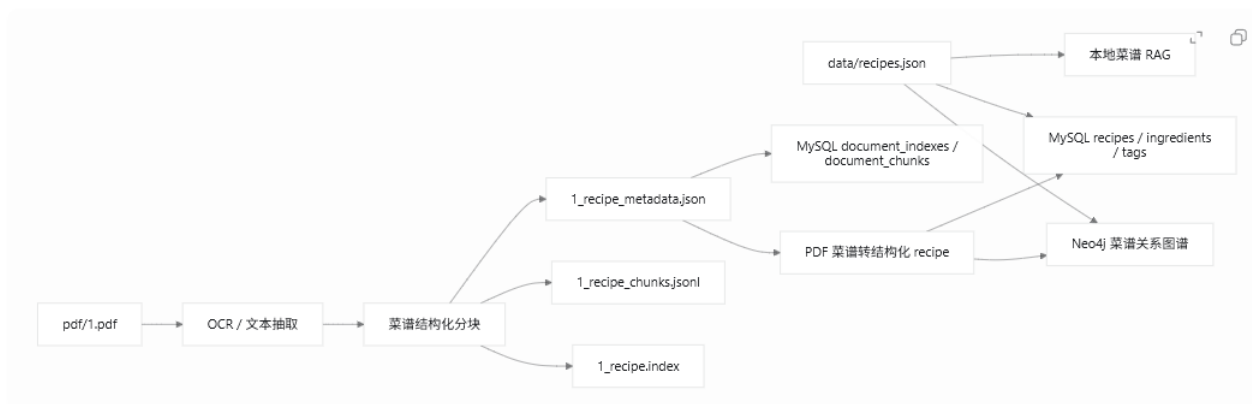
2.2.1 总体架构

图 1.总体架构



2.2.2 数据架构

图 2.数据架构



2.2.3 Agent 状态设计

系统以 AgentState/GraphState 作为共享状态，关键字段包括：

字段	作用
`user_input`	用户原始输入
`session_id`	会话标识，用于记忆读取和写入
`chat_history`	历史对话摘要
`intent`	Router Agent 识别出的意图
`target_agent`	目标 Agent
`retrieved_docs`	RAG 检索候选
`fusion_results`	多源融合结果
`vision_result`	图片识别结果
`agent_output`	当前 Agent 输出
`final_answer`	最终回答
`meta`	调试信息、路由模式、缓存状态、数据来源等

2.3 API Spec

2.3.1 文本聊天接口

项目	内容
方法	`POST`
路径	`/chat`
请求体	`message`, `session_id`, `top_k`
响应	`answer`, `hits`, `intent`, `generator`, `meta`
用途	文本菜谱问答、多轮推荐、SQL/Cypher/Fusion 查询

示例：

[cmd]


```
curl -X POST http://127.0.0.1:8010/chat ^
-H "Content-Type: application/json" ^
-d '{"message\":"推荐低脂高蛋白晚餐\',"session_id\":"demo\',"top_k\:3}'"
```

2.3.2 图片聊天接口

项目	内容
方法	`POST`
路径	`/chat/image`
请求	`multipart/form-data`, 包含 `image`, `message`, `session_id`, `top_k`
响应	图片识别结果、相似菜谱和最终回答
用途	菜品图片识别与多模态菜谱问答

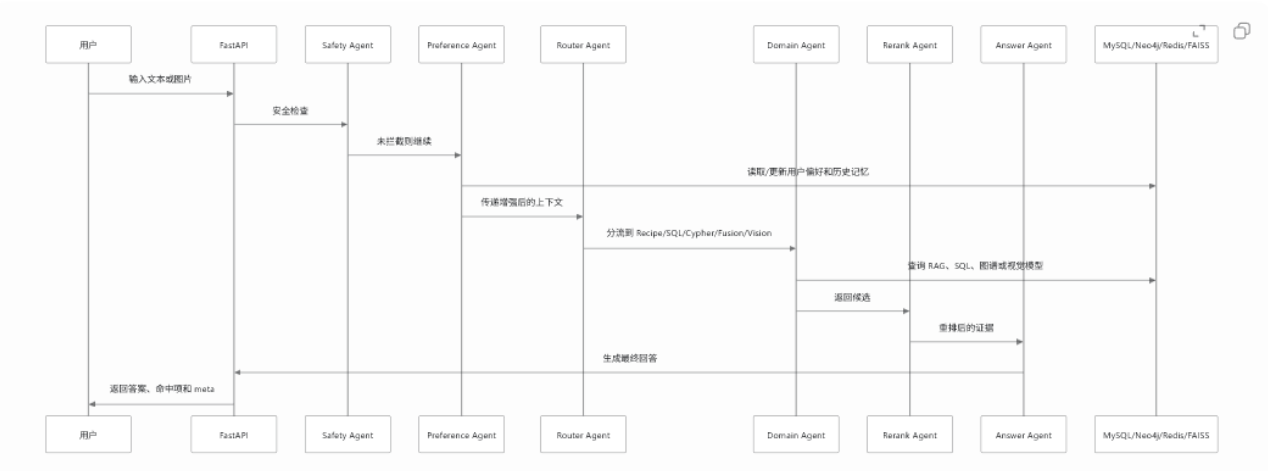
2.3.3 健康检查与调试接口

路径	用途
`/health`	查看服务、检索后端、embedding、memory、database agent 状态
`/debug/stats`	查看 MySQL、Neo4j、Redis 和 retriever 统计
`/debug/evaluation`	查看评估集和能力开关
`/debug/session/{session_id}`	查看指定会话的记忆和偏好
`/ui`	浏览器调试页面
`/ui/database.html`	MySQL / Redis / Neo4j 只读预览页面

三、系统架构与设计

3.1 多 Agent 交互流程

图 3. 多 Agent 交互流程

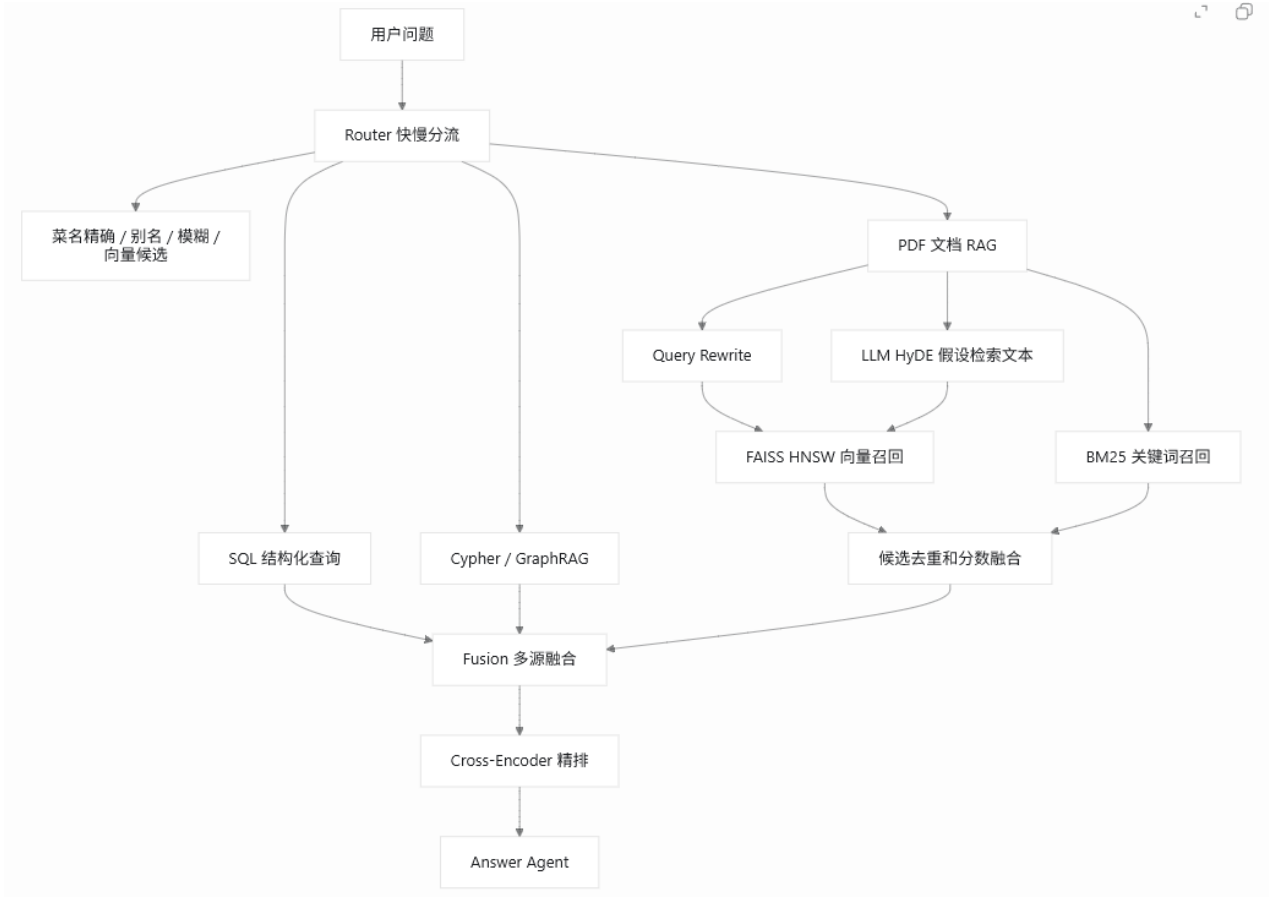


3.2 检索设计

本项目的检索策略采用“召回与精排分离”的设计：

1. Router 先判断问题类型，避免所有请求都进入重检索链路。
2. 简单菜名做法优先走精确匹配，减少“相似菜谱冒充目标菜”的风险。
3. 条件推荐走 SQL、RAG、GraphRAG、Fusion 多源检索。
4. PDF 文档 RAG 使用 Query Rewrite + LLM HyDE + FAISS + BM25 混合召回。
5. Cross-Encoder 使用原始问题进行最终精排，避免扩展查询或 HyDE 文本影响相关性判断。
6. Answer Guard 对本地库空结果、低置信图片识别和证据不足场景进行降级提示。

图 4.检索设计



3.3 存储设计

存储	内容	典型表/文件
JSON	基础菜谱数据	`data/recipes.json`
FAISS	向量索引	`data/processed/1_recipe.index`
MySQL	结构化菜谱、用户偏好、文档 chunk	`recipes`, `ingredients`, `document_chunks`
Neo4j	菜谱-食材-标签-目标关系	`Recipe`, `Ingredient`, `Tag`, `Goal`
Redis	会话历史、偏好、缓存	`session_id` 相关 key
本地 metadata	PDF chunk 元数据	`1_recipe_metadata.json`

3.4 安全与工程规范

- API Key 使用 .env，README 中只提供<DS_KEY>、<MIMO_KEY>占位符。
- SQLAgent 和 CypherAgent 均通过 query guard 限制只读查询。
- 数据库在 Docker 中运行，应用在 conda 环境运行，减少部署复杂度。
- 空结果不缓存，避免错误结果长期污染推荐链路。
- 图片识别失败时使用保守 fallback，不夸大模型能力。

四、关键实现与代码展示

4.1 FastAPI 应用入口

核心文件：app/main.py

系统在 FastAPI 启动时实例化 LLM、Retriever、MemoryStore、SQLAgent、CypherAgent、FusionAgent、VisionAgent，并注入 SmartRecipeGraph。

[python]

```
workflow = SmartRecipeGraph(
    router_agent=RouterAgent(llm_client, enable_database_agents=ENABLE_DATABASE_AGENTS),
    safety_agent=SafetyAgent(),
    preference_agent=PreferenceAgent(memory_store),
    recipe_agent=RecipeAgent(retriever, GraphRAG(cypher_agent.store),
mysql_store=sql_agent.store),
    nutrition_agent=NutritionAgent(retriever),
    sql_agent=sql_agent,
    cypher_agent=cypher_agent,
    fusion_agent=FusionAgent(retriever=retriever, mysql_store=sql_agent.store,
neo4j_store=cypher_agent.store),
    vision_agent=VisionAgent(ImageAnalyzer(vision_client), retriever),
    rerank_agent=RerankAgent(),
    answer_agent=AnswerAgent(llm_client),
    memory_store=memory_store,
)
```

这体现了依赖注入思想：Agent 不直接创建所有外部资源，而是由应用入口统一装配，便于测试和替换。

4.2 LangGraph 工作流

核心文件：app/graph.py

系统使用 StateGraph 构造工作流，先通过 Safety 和 Preference，再由 Router 条件路由到不同 Agent。

[python]

```

graph.set_entry_point("safety")
graph.add_conditional_edges("safety", self._route_after_safety, {
    "preference_agent": "preference_agent",
    "answer_agent": "answer_agent",
})
graph.add_conditional_edges("router", self._route_after_router, {
    "recipe_agent": "recipe_agent",
    "nutrition_agent": "nutrition_agent",
    "sql_agent": "sql_agent",
    "cypher_agent": "cypher_agent",
    "fusion_agent": "fusion_agent",
    "tool_agent": "tool_agent",
    "vision_agent": "vision_agent",
})

```

相比简单 if-else 路由，LangGraph 的优势是状态清晰、节点可观测、可插拔，适合复杂 Agent workflow。

4.3 Router Agent

核心文件：app/agents/router_agent.py

Router Agent 先走规则快速路由，在规则无法确定或需要更复杂判断时再调用 LLM。当前支持的意图包括：

- recipe_search
- recipe_detail
- ingredient_replace
- nutrition_query
- structured_recipe_query
- relationship_query
- multi_source_query
- tool_query
- general_chat
- out_of_scope

这种设计兼顾速度、成本和可靠性：常见问题由规则快速处理，复杂问题再由 LLM 辅助。

4.4 MySQL 与 Neo4j 数据建模

核心文件：app/services/mysql_store.py、app/services/neo4j_store.py

MySQL 表结构包括：

- recipes
- ingredients
- recipe_ingredients
- recipe_tags

- recipe_suitable_for
- user_profiles
- chat_turns
- eval_runs
- document_indexes
- document_chunks

Neo4j 图谱包括:

- 节点: Recipe,Ingredient,Tag,Constraint,Goal,MealTime
- 关系: USES,HAS_TAG,MATCHES,SUITABLE_FOR

这种设计让 SQL 适合精确筛选和统计, Neo4j 适合关系查询和图谱增强。

4.5 PDF 菜谱管线

核心文件: scripts/rebuild_pdf1_pipeline.py

为了满足 PDF 菜谱书导入需求, 系统提供一键管线:

[cmd]

```
python scripts\rebuild_pdf1_pipeline.py --dry-run
python scripts\rebuild_pdf1_pipeline.py
```

该脚本完成:

7. 读取 pdf\1.pdf
8. 强制 OCR
9. 菜谱结构化分块
10. 建立 1_recipe.index
11. 生成 1_recipe_metadata.json
12. 导入 MySQLdocument_indexes/document_chunks
13. 将 PDF 菜谱转为结构化 recipe 后导入 MySQL 和 Neo4j

这体现了 SDD 中“可执行规格”的思想: 文档中定义的数据流最终沉淀为可复现脚本。

4.6 HyDE 与混合检索

核心文件: app/services/hyde.py、scripts/search_document_faiss.py

HyDE 会调用 LLM 生成一个“假设答案文档”, 但该文本只用于向量召回, 不作为最终回答证据。检索输出中会显示:

[json]

```
"hyde": {
  "enabled": true,
  "generator": "llm",
  "hypothetical_document": "..."}
}
```

系统同时使用：

- 原始问题向量召回
- 扩展问题向量召回
- HyDE 文本向量召回
- BM25 关键词召回
- metadata 加权
- Cross-Encoder 精排

4.7 AI IDE 使用说明

本项目开发过程采用 AI 编程助手辅助完成需求拆解、代码生成、错误定位、测试修复和文档整理。AI 辅助主要用于：

- 根据课程要求拆解项目规格。
- 生成 Agent 框架和服务层初稿。
- 根据运行错误定位依赖、路径、数据库和 embedding 维度问题。
- 生成测试脚本、README 和技术文档。
- 辅助完成 PDF RAG、HyDE、GraphRAG 等模块的迭代。

建议在最终 PDF 中补充以下截图：

- docs/assets/ai_ide_spec.png：AI IDE 辅助拆解 Specs 的截图。
- docs/assets/ai_ide_debug.png：AI IDE 辅助调试报错的截图。
- docs/assets/demo_ui.png：/ui 页面运行截图。
- docs/assets/database_ui.png：/ui/database.html 数据库预览截图。

五、测试与评估

5.1 自动化测试

当前项目包含 20 个测试文件，覆盖 RAG、HyDE、GraphRAG、Fusion、Vision、Debug API、Docker 配置、数据集和工具 Agent 等模块。

测试命令：

```
[cmd]
python -m pytest tests -q -p no:cacheprovider
```

本次在当前工作区执行结果：

```
[text]
91 passed in 6.00s
```

5.2 测试覆盖范围

测试文件	覆盖内容
`test_retriever_hybrid.py`	混合检索、HyDE 召回合并
`test_hyde.py`	HyDE 生成和 LLM 不可用降级
`test_graph_rag.py`	Neo4j GraphRAG 上下文增强
`test_fusion_agent.py`	RAG / SQL / Cypher 多源融合
`test_vision_pipeline.py`	图片识别链路和 fallback
`test_document_chunking.py`	文档读取和 chunk 逻辑
`test_recipe_chunk_refiner.py`	PDF 菜谱结构化解析
`test_tool_agent.py`	Tool Agent 规划和工具调用
`test_debug_api.py`	Debug API
`test_database_browser_api.py`	数据库只读预览接口
`test_llm_query_generation.py`	Text2SQL / Text2Cypher 生成控制
`test_ablation.py`	消融配置和评估说明

5.3 功能验证用例

用例	输入	期望结果
文本推荐	“推荐低脂高蛋白晚餐”	Router 进入 Recipe/Fusion，返回符合目标的菜谱
菜谱详情	“老醋花生米怎么做”	优先从 PDF 文档索引或结构化库返

		回做法
偏好记忆	“我不吃香菜，对虾过敏”	Preference Agent 写入记忆，后续推荐避开相关食材
SQL 查询	“热量最低的五道菜”	SQL Agent 生成只读 SELECT 并返回排序结果
图谱查询	“鸡胸肉通常和什么搭配”	Cypher Agent 查询 Neo4j 食材共现关系
图片识别	上传菜品图片	Vision Agent 识别菜品并检索相似菜谱

图 5.示例 1 “我不吃香菜，对虾过敏，给我推荐低脂高蛋白晚餐”



图 6.示例二 “老醋花生米怎么做”



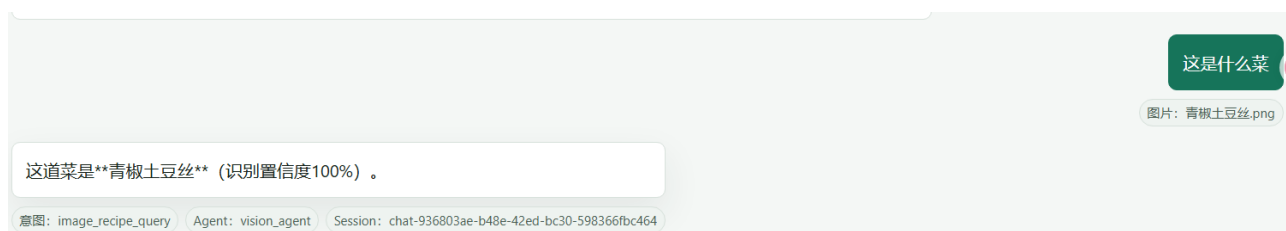
图 7.示例三 “热量最低的五道菜”



图 8.示例四 “鸡胸肉通常和什么搭配”



图 9.示例五 “这是什么菜”



5.4 评估脚本

项目提供多种评估脚本：

[cmd]

```
python scripts/evaluate_retrieval.py
python scripts/evaluate_router.py
python scripts/evaluate_text2sql.py
python scripts/evaluate_text2cypher.py
python scripts/evaluate_document_rag.py
python scripts/run_ablation.py
```

消融实验关注以下问题：

- 关闭 GraphRAG 后，图谱上下文对答案是否有贡献。

- 关闭 Fusion 后，多源融合是否优于单一路径。
- 只保留 RAG 时，与完整数据库增强 Agent 相比是否退化。
- 不同检索后端对 hit@k 的影响。

5.5 可观测性

系统在每个 LangGraph 节点执行后输出中文日志，包含 Agent 名称、耗时、意图、目标 Agent 和输出摘要。FastAPI 中间件记录请求路径、状态码和耗时，并通过 X-Process-Time-Ms 返回处理时间。Debug API 可查看 MySQL、Neo4j、Redis、retriever 和评估数据状态。

六、系统升级与扩展

6.1 当前不足

- 14. PDF OCR 对扫描质量、版式和编码较敏感，复杂菜谱书仍需增强清洗规则。
- 15. HyDE 依赖外部 LLM，可用性受 API 配置和网络影响。
- 16. 当前系统以本地开发为主，尚未部署到云服务器提供公网访问。
- 17. 视觉识别虽然已接入，但缺少大规模图片评测集。
- 18. AI IDE 截图、Demo 视频和最终 PDF 书签仍需在提交前补齐。

6.2 下一阶段计划

阶段	目标	方案
V1	提高 PDF RAG 稳定性	增强 OCR 后处理、菜名纠错和 metadata 校验
V2	引入更完整的工具协议	将内部工具注册表升级为标准 Function Calling 或 MCP 接口
V3	云端部署	使用 Docker Compose / 云服务器部署 FastAPI 与数据库
V4	LLMOps 可观测性	加入 tracing、prompt 版本管理和自动评测面板
V5	个性化推荐增强	引入用户长期画像、菜谱反馈和推荐解释

6.3 AI 能力演进路径

未来 SmartRecipe 可以从“问答助手”演进为“饮食规划 Agent”：

- 根据用户健康目标自动生成一周菜单。
- 结合冰箱库存和预算生成购物清单。
- 根据烹饪设备约束推荐做法。
- 自动读取菜谱书、网页和视频字幕，更新知识库。
- 为家庭成员生成差异化饮食方案。

七、课程总结

7.1 个人收获

通过本项目，我对“从代码编写者到智能体编排者”的转变有了更具体的理解。传统软件开发更关注接口、数据库和业务逻辑，而 Agentic AI 应用需要额外关注模型能力边界、工具调用、状态流转、记忆管理、检索证据和评估闭环。

本项目的最大收获包括：

19. 理解了 LangGraph 状态机在多 Agent 编排中的价值。
20. 认识到 RAG 不是简单向量搜索，而是包含数据清洗、切块、召回、精排、证据约束和降级策略的一整套工程体系。
21. 掌握了 MySQL、Neo4j、Redis、FAISS 在智能应用中的不同职责。
22. 体会到评估和可观测性的重要性。没有测试、日志和 Debug API，就很难判断 Agent 是否真的按预期工作。
23. 认识到 AI 编程助手适合加速实现，但关键架构决策和事实校验仍必须回到代码和运行结果本身。

7.2 工程思维转变

在项目早期，容易把“大模型回答”当成系统核心。但随着功能增加，我逐渐意识到企业级 AI 应用的核心并不是单次生成，而是“可控的工作流”。一个可靠的 Agent 系统需要：

- 明确的输入输出规格。
- 可解释的路由逻辑。
- 可复现的数据处理脚本。
- 可替换的模型和检索后端。

- 可观测的状态和日志。
- 可执行的测试与评估。
- 对不确定结果的保守处理。

7.3 对课程的建议

- 建议课程继续保留“真实工程闭环”的大作业形式，同时可以增加以下内容：
 - 提供 Agent 项目的参考评估模板，例如 Router 准确率、RAG hit@k、工具调用成功率。
 - 增加一次关于 MCP / Function Calling / Tool Use 差异的专题实践。
 - 增加 LLMOps 与 tracing 的示例，让学生更好地理解生产级 Agent 的调试方式。
 - 鼓励学生展示失败案例和修复过程，因为 Agent 系统的工程价值往往体现在边界处理和调试中。